Module 7

I²C — Protocol + RTL + Basic Testbench

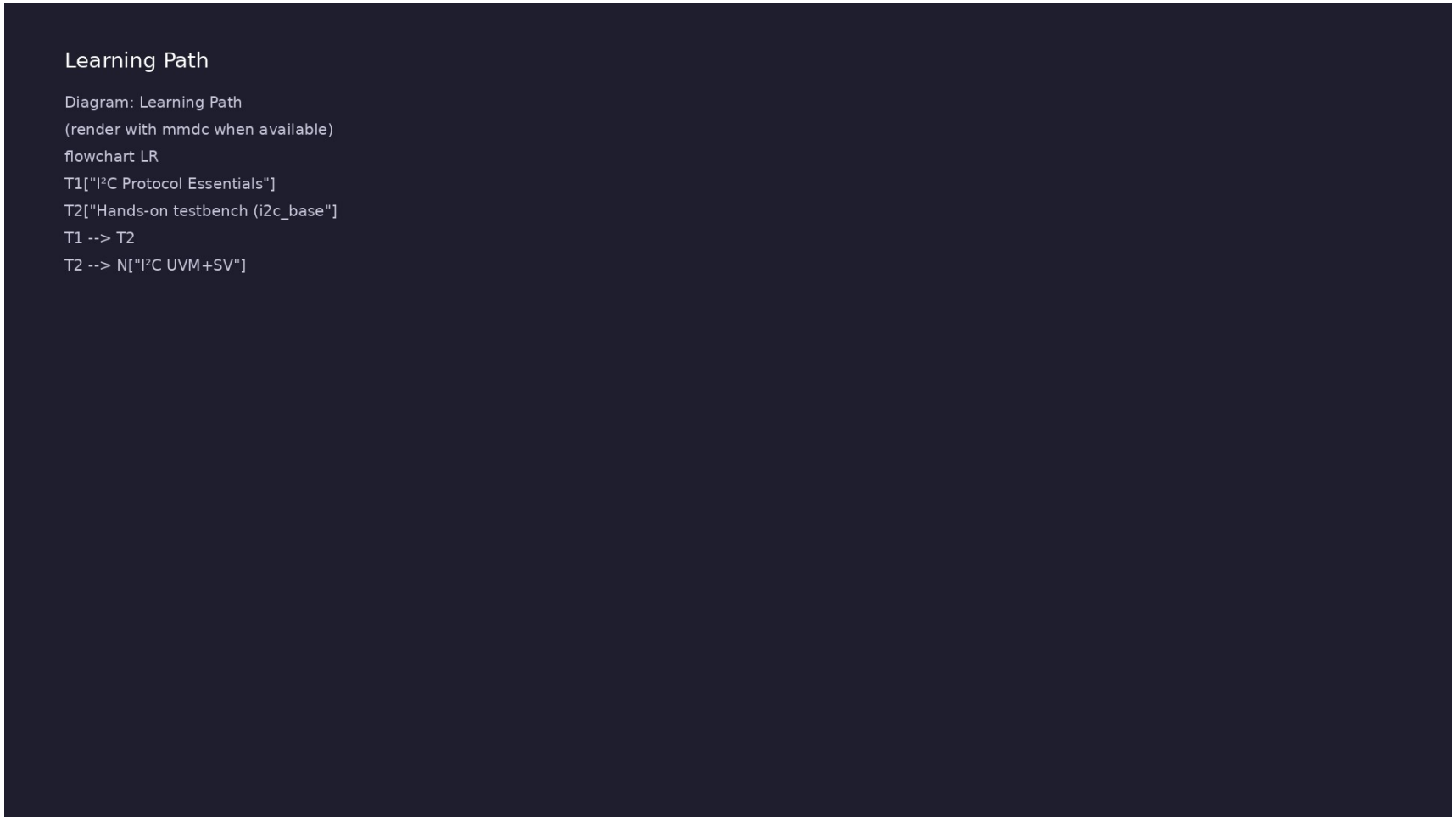
Learning objectives

- RTL block architecture: `clk_div` + `i2c_master` FSM (START → addr → data → STOP).
- Baseline verification: Directed writes plus inline bus monitor (sample SDA on rising SCL).
- Protocol vs simplification: Real open-drain/ACK vs this repo’s push-pull teaching model.
- Path to Module 8: Same DUT with UVM monitor/scoreboard on address and data phases.

Prerequisites

- Modules 1–6.
- Verilator, Make, C++ compiler. No UVM required for this baseline.

Learning path



Understand the I²C protocol (start/stop, addressing, ACK/NACK), translate it to RTL (simple master + timing), and verify with a **basic** (…

Overview

- Module 7 is the I²C **protocol + RTL + basic testbench** module (UVM comes in Module 8).

How to learn this module

Suggested learning path

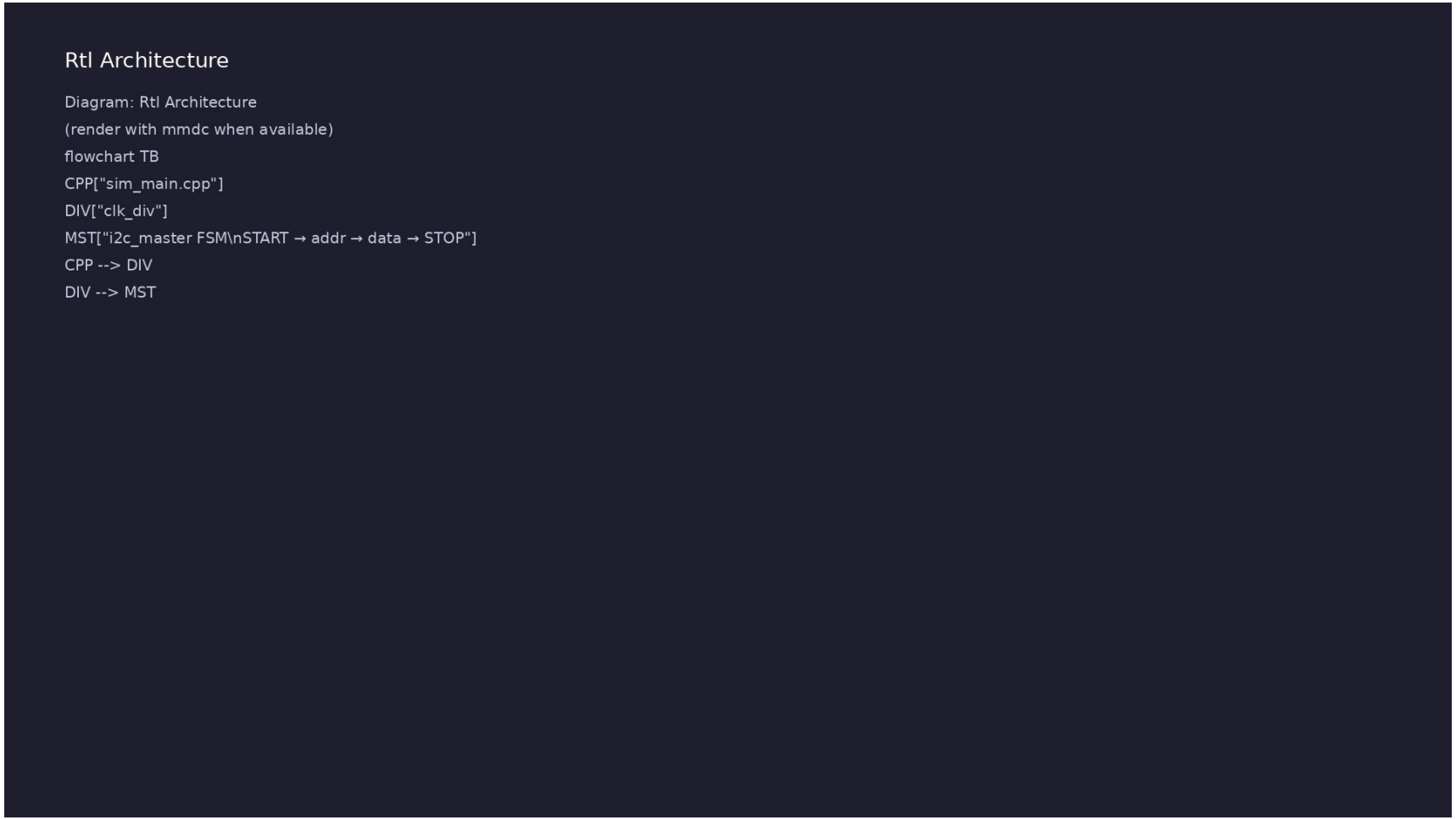
- Read the detailed I²C learning guide:

  I2C_LEARNING_GUIDE.md — what I²C is, what kind of protocol (s…
- Read the protocols + UVM overview:

  LEARNING_GUIDE_PROTOCOLS_AND_UVM.md — Part B § 4. When to Use Ba…

Design architecture

RTL / block architecture



Module 7: DUT / block hierarchy (see module7/examples/).

Verification environment architecture

Module 7: UVM layers, agents, and checkers for this phase.

1. Block hierarchy

- top_i2c_baseline → `clk_div` + `i2c_master`; C++ drives `clk`/`rst_n`.

2. I²C master FSM

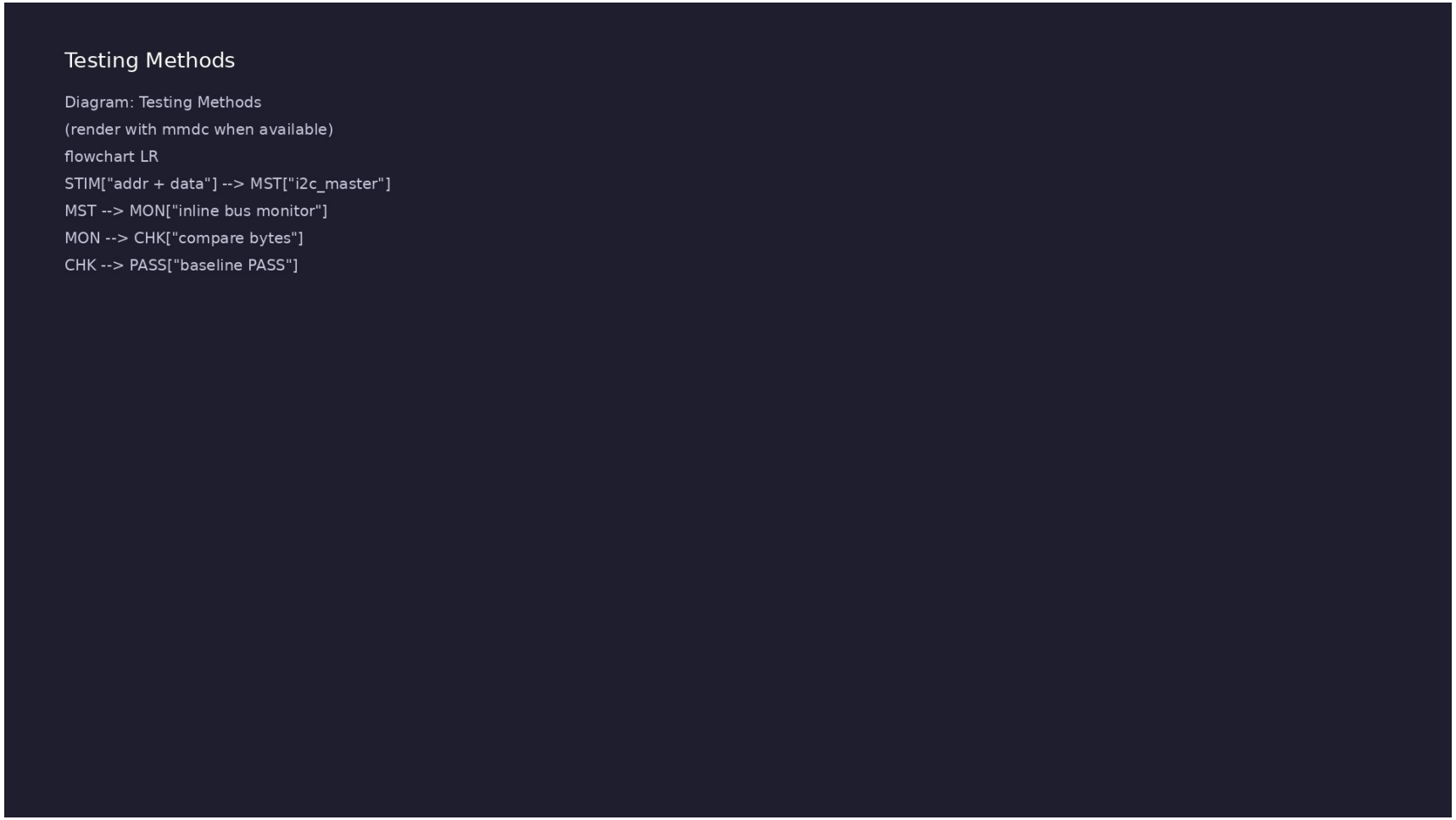
- States: IDLE → START → ADDR_BITS → DATA_BITS → STOP.
- One teaching write: 7-bit `addr` + data byte on
  `scl`/`sda` (push-pull model).

3. Teaching vs real I²C

- No open-drain, ACK/NACK, or arbitration in baseline
  — learn sequencing first.

Verification & testing methods

Testing and closure flow



Module 7: how tests, checks, and metrics close verification goals.

1. Inline bus monitor

- Top module detects START; samples SDA on rising SCL;
  rebuilds addr+W and data.

2. Self-check flow

- Compare captured bytes to driven `addr`/`data_in`;
  print `I2C baseline test PASS`.

3. Path to Module 8

- Move monitor into I2cMonitor; scoreboard splits address and data phases.

Syllabus topics

1. I²C Protocol Essentials

- Bus idle: SCL=1, SDA=1.
- START: SDA goes 1→0 while SCL is 1.
- STOP: SDA goes 0→1 while SCL is 1.
- Address phase: 7-bit address + R/W bit (0=write, 1=read) sent MSB first.
- Data phase: 8-bit data bytes sent MSB first.
- ACK/NACK (concept): receiver pulls SDA low for ACK on the 9th clock; high means NACK.

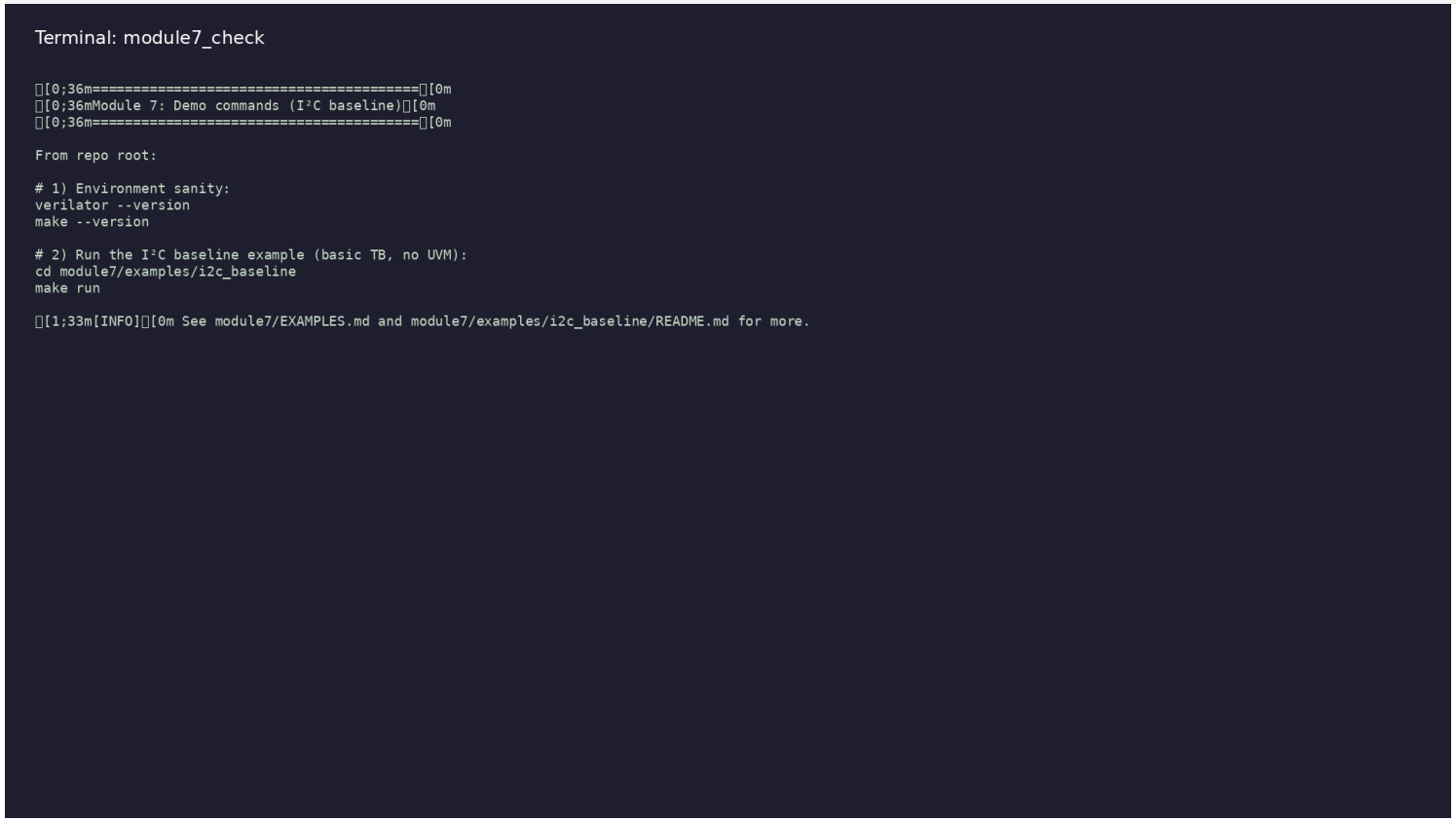
2. Hands-on testbench (i2c_baseline)

- Stimulus: `start`, `addr`, `data_in`, `wait(done)`.
- Monitor/self-check: reconstruct address+W and data
  from SCL/SDA; compare to expected.

Hands-on examples

Module 7 self-check

```
$ ./scripts/module7.sh --help
```

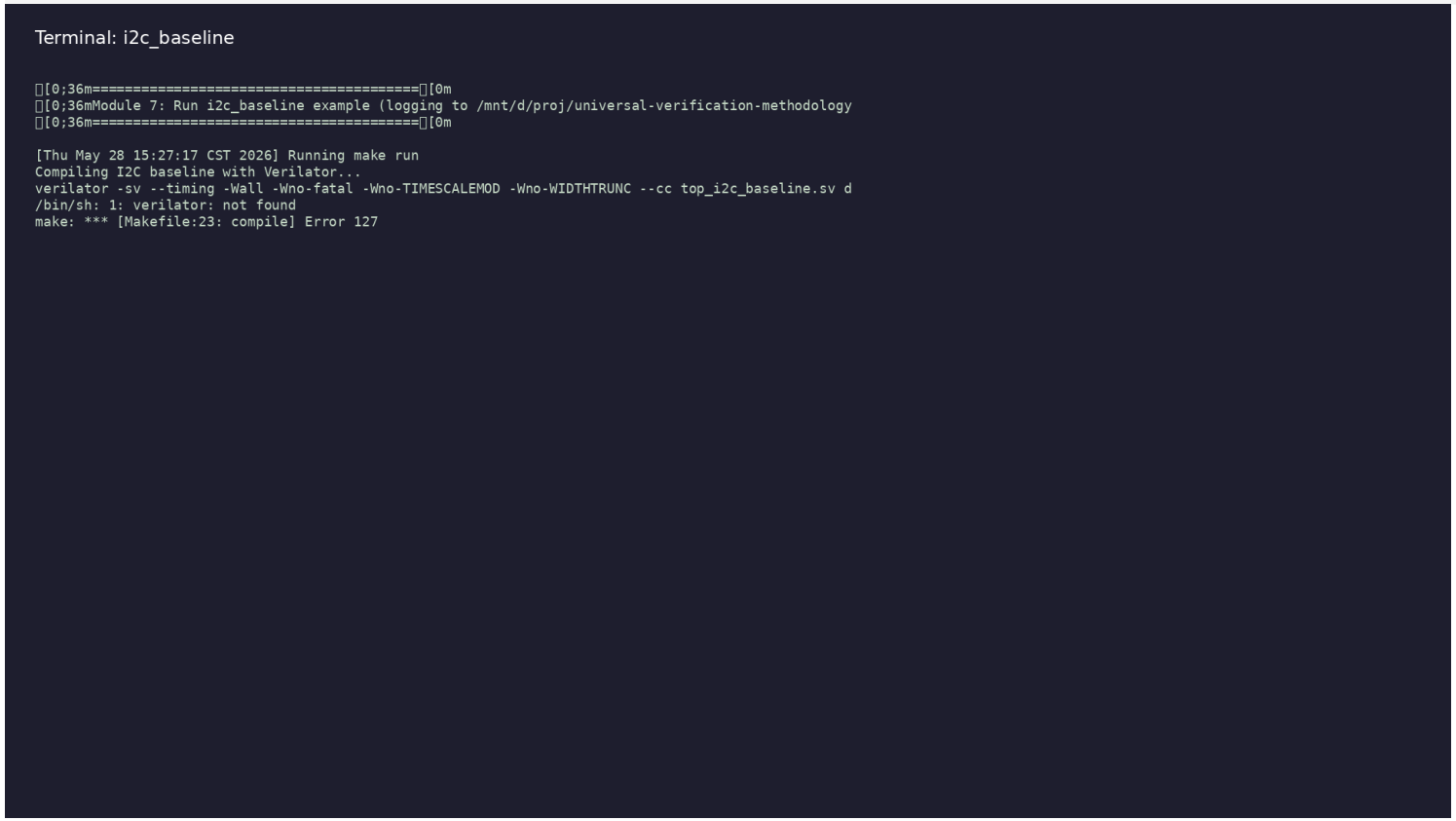


Example 1: I²C baseline

- Drives a single write transaction (START → address+W → data → STOP)
- Includes a simple bus monitor that samples SDA on SCL rising edges to reconstruct bytes
- No UVM

Demo: I²C baseline

```
$ ./scripts/module7.sh --run
```



Practice & assessment

Exercises

- Run i2c_baseline
- Trace one transfer
- Bus monitor
- Optional: Waveforms

Assessment checklist

- Can describe I²C START, STOP, address+R/W, and data phase (and ACK/NACK concept).
- Can explain the role of i2c_master and clk_div; know the baseline uses push-pull (simplified).
- Can run i2c_baseline and interpret the test result.
- Ready for Module 8: I²C UVM+SV verification.

Summary & next steps

- Understand the I²C protocol (start/stop, addressing, ACK/NACK), translate it to RTL (simple master…
- Complete module7/CHECKLIST.md
- Review module7/EXAMPLES.md and run each lab
- Next: I²C UVM+SV